Designing and Building a Computer Science Portfolio

Uygar Tuna Oflas
B201 Computer Science Lab — Gisma University of Applied Sciences

June 26, 2026

Live site: https://tunoflas1.github.io/cs-portfolio1/ | **Repository:** https://github.com/tunoflas1/cs-portfolio1

1 Introduction

This report documents the design, implementation and content of a personal computer-science portfolio, produced for the B201 Computer Science Lab. The brief frames the task realistically: a hiring manager for a working-student position has asked for a portfolio that demonstrates technical skill. The deliverable is therefore two connected things — a portfolio website hosted on GitHub Pages, and the repository behind it — both of which should be clear, well structured and honest about what they show.

My guiding principle was *restraint with intent*. As an early-career student, I do not have a long list of large projects, so I chose to present a small number of focused exercises, each demonstrating one idea cleanly, rather than padding the site. The report explains how the site and repository are organised, justifies the main design decisions, lists the tools used, and reflects critically on the result.

2 Structure of the Portfolio

2.1 Website

The website is a single page divided into labelled sections that a recruiter can scan in order: a **hero** introduction, **skills**, an **interactive demo**, **projects**, **experience**, **education** and **contact**. A single-page layout suits a portfolio of this size: the visitor sees everything by scrolling, and a sticky navigation bar provides quick jumps without page reloads. This follows the common guidance that navigation should let users locate information with minimal effort (Nielsen, 2020).

The hero states who I am and what I am looking for in one sentence, because the top of a page carries the most weight in how quickly visitors form a judgement (Lidwell et al., 2010). Below it, four short "fact" chips (location, studies, languages, interests) give context at a glance.

2.2 Repository

The repository mirrors the site's content but is organised for reading code rather than browsing:

```
cs-portfolio/
  index.html            # the website (HTML + CSS + JS in one file)
  cv/                   # LaTeX CV source and compiled PDF
  projects/             # one folder per technical exercise
    knight_path/        # Python BFS + unit tests
    text_insights/      # Python CLI
    guestbook/          # PHP endpoint
  report/               # this report (LaTeX + PDF)
  README.md             # overview and run instructions
```

Each project lives in its own folder so it can be read in isolation, and the `README.md` acts as a map with run instructions. Keeping a clear, predictable project layout is a widely recommended habit because it lowers the effort needed to understand a codebase (Wilson et al., 2017).

3 Design Decisions

3.1 A single static page, no framework

I deliberately built the site with plain HTML, CSS and JavaScript rather than a framework such as React. For a static portfolio with no dynamic data, a framework would add a build step and dependencies without adding value; the simplest tool that does the job is the right one. GitHub Pages serves static files directly (GitHub, 2024), so a single `index.html` deploys with no configuration. Inlining the CSS and JavaScript keeps the whole site in one reviewable file, which is reasonable at this scale, though I note the trade-off in the reflection.

3.2 Visual identity

Most developer portfolios look alike, so I made specific choices rather than accepting defaults. The palette is a cool light-grey paper (`#ECEFF3`) with a single indigo accent (`#2C44D6`); limiting the design to one accent colour keeps attention on the content and avoids visual noise (Lidwell et al., 2010). The type pairing is intentional: *Space Grotesk* for headings gives a technical character, *Inter* for body text is highly legible at small sizes, and *JetBrains Mono* is used only for labels and code, where a monospace face signals "this is technical". Pairing a display face with a separate, readable body face is a standard typographic principle (Butterick, 2019).

3.3 The interactive demo as a signature

Rather than a static screenshot, the centre of the page is a working **knight's-shortest-path** board. Clicking a start square and a target square runs a breadth-first search in the browser and draws the optimal route. I chose this for three reasons: it ties to a personal interest (chess), it shows a real algorithm rather than describing one, and it connects directly to the Python implementation in the repository, so the demo and the code reinforce each other. A small interactive moment also demonstrates DOM manipulation and event handling without a framework.

3.4 Accessibility and responsiveness

The site is built to a basic quality floor. Layouts use responsive CSS grids that collapse to a single column on narrow screens, so the page is usable on a phone. Colour contrast between text and background was kept high for readability, in line with WCAG guidance (W3C, 2018). Interactive controls are real `<button>` elements with `aria-labels` and visible keyboard focus styles, and all motion is disabled when the visitor's system requests reduced motion (`prefers-reduced-motion`). These are small additions but they reflect the principle that an interface should work for as many people as possible (W3C, 2018).

3.5 The CV in LaTeX

The brief requires a LaTeX CV, and LaTeX is a good fit: it produces consistent, typeset PDFs and separates content from formatting (Lamport, 1994). I wrote a custom one-column layout using only standard packages, so it compiles anywhere without extra installation, and reused the same indigo accent as the website so the CV and site read as one identity.

4 Tools and Technologies

- **HTML, CSS, JavaScript** — the website, with no framework or build step.
- **GitHub Pages** — free static hosting served from the repository (GitHub, 2024).
- **Git & GitHub** — version control and the public repository.
- **LaTeX** — the CV and this report.

- **Python** — the knight-path finder (with `unittest`) and the text-insights CLI.
- **PHP** — the guestbook endpoint sample.
- **Google Fonts** — Space Grotesk, Inter and JetBrains Mono.

5 Implementation of the Technical Exercises

5.1 Knight Path Finder (Python + JavaScript)

The core exercise finds the fewest knight moves between two squares. It models the board as an implicit graph where each square connects to the squares a knight can jump to, and runs a breadth-first search. BFS is the right choice because it explores the graph in order of distance, so the first time it reaches the target it has used the minimum number of moves (Cormen et al., 2009). The Python version is the reference implementation and is covered by unit tests, including the known result that a corner-to-corner trip (a1 to h8) takes six moves. The JavaScript version on the website uses the same algorithm so the visible demo and the tested code agree.

5.2 Text Insights CLI (Python)

A command-line tool that reads a text file and reports word and sentence counts, average lengths, and the most frequent content words after removing common stop words. It is a small exercise in parsing, using dictionaries (via `collections.Counter`) and presenting tidy output — everyday tasks that appear constantly in real work.

5.3 Guestbook API (PHP)

A minimal endpoint that accepts short messages and stores them to a JSON file. The point of the exercise is not the storage but the discipline around input: every field is checked for presence, type and length, and text is escaped before it is stored, which guards against the kind of cross-site-scripting issue that arises when user input is trusted. Validating and encoding untrusted input is a basic security expectation (OWASP, 2021). PHP runs on a server rather than on GitHub Pages, so this file is included as a readable sample rather than a live feature.

6 Reflection: Strengths, Weaknesses and Future Work

6.1 Strengths

The portfolio is honest and coherent. It does not overstate experience; instead it shows a small set of working, documented exercises, and the interactive demo proves a claim rather than asserting it. The site and CV share one visual identity, the repository is tidy and explained, and the whole thing deploys with no build step, which keeps it easy to maintain.

6.2 Weaknesses

There are real limits. Inlining the CSS and JavaScript into one HTML file is convenient at this size but would become hard to maintain as the site grows; splitting them into separate files would be the next step. The project set is deliberately small and leans toward classic exercises rather than a larger, original application. There are no automated tests for the front-end JavaScript, and accessibility has been addressed by hand rather than verified with an audit tool.

6.3 Future work

Given more time I would: (1) separate the stylesheet and script and add a light build or linting step; (2) add a larger original project — for example a small full-stack application — to show work beyond isolated exercises; (3) run an accessibility audit (e.g. Lighthouse) and act on its findings;

and (4) add a short "writing" or "notes" section, since the ability to explain technical work clearly is itself a skill worth showing. These changes would deepen the portfolio without changing its core principle: show real, understood work, clearly.

References

References

Butterick, M. (2019) *Practical Typography*. 2nd edn. Available at: https://practicaltypography.com (Accessed: 26 June 2026).

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*. 3rd edn. Cambridge, MA: MIT Press.

GitHub (2024) *GitHub Pages Documentation*. Available at: https://docs.github.com/en/pages (Accessed: 26 June 2026).

Lamport, L. (1994) *LaTeX: A Document Preparation System*. 2nd edn. Reading, MA: Addison-Wesley.

Lidwell, W., Holden, K. and Butler, J. (2010) *Universal Principles of Design*. Beverly, MA: Rockport.

Nielsen, J. (2020) *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/ (Accessed: 26 June 2026).

OWASP (2021) *OWASP Top 10: Web Application Security Risks*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 26 June 2026).

W3C (2018) *Web Content Accessibility Guidelines (WCAG) 2.1*. Available at: https://www.w3.org/TR/WCAG21/ (Accessed: 26 June 2026).

Wilson, G. et al. (2017) 'Good enough practices in scientific computing', *PLOS Computational Biology*, 13(6).